

HD_ACS 설치 매뉴얼

단계별 설치·기동·검증 절차

문서 버전	1.0 (2026-09-02)
대상	HD_ACS를 PC(개발/실험실/현장 서버)에 설치·기동하려는 담당자
소요 시간	인프라 포함 약 30~60분 (다운로드 제외)
구성 요소	HD.Acs.App(서버 :5199) · HD.Acs.UI(WPF) · PostgreSQL(:5432) · MQTT 브로커(:1883) · Simulator(검증용)
원본	docs/INSTALL_MANUAL.md (git이 truth — 본 PDF는 스냅샷)

개발 PC에는 2~3단계 인프라(docker dev-postgres/dev-rabbitmq)가 이미 구동·구성돼 있다 — 개발 PC 기준으로는 **5단계(설정)부터** 진행. 신규 PC는 0단계부터 순서대로.

0단계. 사전 요구사항

항목	요구	확인
OS	Windows 10/11 (UI 필수 — WPF). 서버·시뮬레이터만이면 macOS/Linux 가능	—
.NET SDK	8.0 이상 (9.x로 8.0 타킷 빌드 가능)	<code>dotnet --version</code>
Docker Desktop	인프라를 docker로 쓸 경우 (권장)	<code>docker --version</code>
Git	소스 확보용	<code>git --version</code>
Telerik 계정	UI 빌드 시에만 — nuget.telerik.com 자격증명	4단계

⚠ **포트 확인:** 서버 포트는 **5199**다 — 5100은 개발 PC의 상주 제품(CS01_Pexe)과 충돌하여 회피한 관례 포트. 대상 PC에서 5199-5432-1883이 비어 있는지 확인:

```
Get-NetTCPConnection -LocalPort 5199,5432,1883 -State Listen -ErrorAction SilentlyContinue
```

1단계. 소스 확보

```
git clone https://github.com/gony0811/HD_ACS.git D:\Github\HD_ACS
cd D:\Github\HD_ACS
```

폐쇄망이면 개발 PC에서 리포 폴더 전체 복사 (bin/ , obj/ 제외 가능).

2단계. 인프라 기동 — PostgreSQL + MQTT 브로커

방법 A — Docker Compose (권장, 개발 PC 방식)

```
cd D:\Github\HD_ACS\docker
# .env 확인/작성: POSTGRES_USER=postgres POSTGRES_PASSWORD=postgres POSTGRES_DB=hdacs
# RABBITMQ_USER=... RABBITMQ_PASSWORD=...
docker compose up -d
# dev-postgres(5432) · dev-rabbitmq(1883/5672/15672) 확인
docker ps
```

- PostgreSQL 볼륨이 비어 있는 최초 기동이면 `db/schema.sql` 이 자동 적용된다(3단계 생략 가능).
- RabbitMQ는 MQTT 플러그인이 활성화돼 1883에서 MQTT를 서비스한다.

방법 B — 네이티브 설치 (현장 서버, ADR-011)

- PostgreSQL 16 설치 → 서비스 자동 시작
- Mosquitto** 설치 → `mosquitto.conf` 에 `listener 1883` + `allow_anonymous true` (폐쇄망 전제) → OS 서비스 등록

△ 실 AMR이 접속하는 환경이면 브로커가 localhost가 아닌 **PC의 실제 IP**로 수신하도록 하고 방화벽에서 1883 인바운드 허용.

3단계. DB 스키마 적용 (방법 A 최초 기동이면 생략)

```
# docker (DB가 없을 때만 CREATE - 이미 있으면 "already exists" 오류는 무해)
docker exec dev-postgres psql -U postgres -c "CREATE DATABASE hdacs;"
docker cp D:\Github\HD_ACS\db\schema.sql dev-postgres:/tmp/schema.sql
docker exec dev-postgres psql -U postgres -d hdacs -f /tmp/schema.sql

# 네이티브
createdb -U postgres hdacs
psql -U postgres -d hdacs -f D:\Github\HD_ACS\db\schema.sql
```

schema.sql 은 **시드 포함 통합 최신본**: 30테이블+2뷰, startWeldInspection 액션 카탈로그(u,v 포함), 알람 코드 6 중, 로봇 AMR-01(HHI/AMR-01).

기존 DB 업그레이드에만 db/migrations/ 를 날짜순 적용(전부 idempotent). 신규 설치는 schema.sql 하나로 끝.

```
# 확인
docker exec dev-postgres psql -U postgres -d hdacs -c "SELECT robot_id, manufacturer, serial_number FROM
ref.robot;"
# → AMR-01 | HHI | AMR-01
```

4단계. 빌드

4.1 서버·시뮬레이터 (Telerik 불필요)

```
cd D:\Github\HD_ACS\src
dotnet build HD.Acs.App\HD.Acs.App.csproj
dotnet build HD.Acs.Simulator\HD.Acs.Simulator.csproj
# 단위 테스트 53건 통과 확인
dotnet test HD.Acs.Core.Tests\HD.Acs.Core.Tests.csproj
```

4.2 UI (Windows 전용 · Telerik 자격증명 필요)

```
dotnet nuget update source Telerik --username <계정 메일> --password <비밀 번호> --store-password-in-clear-text
dotnet build HD.Acs.UI\HD.Acs.UI.csproj
```

⚠ 루트 nuget.config에 **packageSourceMapping**을 추가하지 말 것 — Telerik.Licensing=nuget.org / MediaFoundation=Telerik 피드로 나뉘어 매핑이 복원을 깨뜨린 이력. 폐쇄망은 개발 PC의 %USERPROFILE%\..nuget\packages 캐시 복사로 오프라인 복원.

5단계. 설정 확인 — src/HD.Acs.App/appsettings.json

키	기본값	변경이 필요한 경우
ConnectionStrings:Default	localhost:5432 / hdacs / postgres	DB가 다른 호스트/계정일 때
Acs:Mqtt:Host / Port	localhost / 1883	브로커가 다른 호스트일 때
Acs:Api:ListenHost	localhost	다른 PC의 UI가 접속하면 0.0.0.0으로 (방화벽 5199 인바운드 허용)
Acs:Api:ListenPort	5199	유지 권장 (5100 금지)
Acs:Area:StationStandoffM	0.8	정차 이격 — 로봇 치수 확정(N10) 시
Acs:Dispatch:MaxRetries	2	실패 재시도 상한
Acs:Dispatch:AllowedDevXy / Theta	0.08 / 0.07	도착 판정 허용 오차

UI: src/HD.Acs.UI/appsettings.json 의 Acs:BaseUr1 = http://localhost:5199 확인. 서버가 다른 PC면 ① 서버 ListenHost = 0.0.0.0 ② UI BaseUr1 = http://<서버IP>:5199 (REST-SignalR 공용). 수정 후 UI 재빌드 또는 bin의 appsettings 직접 수정.

6단계. 서버 기동·확인

```
cd D:\Github\HD_ACS\src
# 또는 bin\Debug\net8.0\HD.Acs.App.exe 직접 실행 (어느 폴더에서든 가능)
dotnet run --project HD.Acs.App
```

기동 로그 확인: `Now listening on: http://localhost:5199` · VDA 5050 마스터 기동 - 로봇 1대 구독

```
Invoke-RestMethod http://localhost:5199/api/robots
# (Mac/Linux) curl http://localhost:5199/api/robots
# → AMR-01 응답이면 서버·DB 정상
```

7단계. UI 기동

```
dotnet run --project HD.Acs.UI
```

- 상단 연결 칩 **연결됨**(SignalR) 확인, 운영/계획/이력 모드 탭 표시 확인
- 계획 데이터가 없으면 **파일** ▶ **새 프로젝트**로 선창 파라미터 등록부터 (운영 방법은 MANUAL 참조)

8단계. 시뮬레이터로 설치 검증 (실로봇 불필요)

```
cd D:\Github\HD_ACS\src
# 무인자 = localhost / HHI / AMR-01 / CT1-L1
dotnet run --project HD.Acs.Simulator
```

- 콘솔 `[SIM] AMR-01 ONLINE` + UI 로봇 상태 `ONLINE` = **MQTT 사슬 정상**
- (계획 데이터가 있다면) 시나리오 선택 → 미션 시작 → 작업 현황·지도 상태색 → `run COMPLETED` = 전체 설치 검증 완료
- 자동 검증: `cd src && ./run_simtest.sh` (bash — 계약 3시나리오 PASS)

9단계. (선택) OS 서비스 등록 — 현장 상시 운영 [ADR-011]

전용 배포 스크립트는 아직 없다(후속 과제 Q4). 아래는 수동 절차.

```
# 1) 게시
dotnet publish src\HD.Acs.App -c Release -o C:\HDACS\app

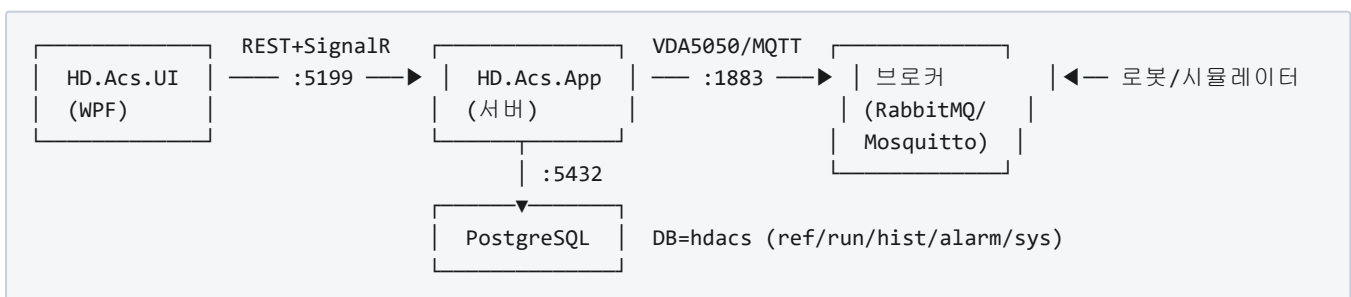
# 2) Windows 서비스 등록 (관리자 PowerShell)
sc.exe create HDACS-App binPath= "C:\HDACS\app\HD.Acs.App.exe" start= auto
sc.exe start HDACS-App
```

- App은 content root가 exe 위치로 고정돼 있어 서비스(cwd=System32)에서도 appsettings를 정상 로드
- Mosquitto·PostgreSQL도 서비스 자동 시작 — 기동 순서 의존 없음(App이 브로커 연결 5초 간격 재시도)

10단계. 문제 해결 (실제 발생 이력 기반)

증상	원인 / 조치
API 호출이 404·영동한 응답	다른 프로그램의 포트 점유 — 개발 PC의 5100은 CS01_P. <code>Get-NetTCPConnection -LocalPort <포트></code> 로 점유 확인. HD_ACS는 5199
기동 시 ConnectionString not initialized	구버전 빌드에서 exe를 타 폴더 실행 — 최신 소스 재빌드(content root 고정 반영)
기동 시 address already in use	이전 App 인스턴스 잔존 — <code>Get-Process HD.Acs.App Stop-Process</code> 후 재기동
시뮬레이터/로봇 ONLINE 안 됨	serial 3자 불일치 — ref.robot.serial_number = 로봇 serial = MQTT 토픽 요소 일치 필수 (기본 AMR-01)
등록/조회 500	DB 스키마 구버전 — 3단계 schema.sql(또는 migrations 순서 적용) 재확인
UI NuGet 복원 실패	Telerik 자격증명 미설정 / packageSourceMapping 추가됨 — 4.2 참고
UI가 구버전 동작	재빌드 없이 구 exe 실행 — <code>dotnet build</code> 후 재실행
run 시작 409	같은 로봇의 진행 중 run 존재 — "이어하기" 또는 "중단" 후 시작

부록. 구성 요소·포트 요약



관련 문서: 운영 방법 MANUAL.md · 운영 사양 ACS_OPERATION_SPEC.pdf · 로봇 계약 VDA5050_INTERFACE_SPEC.pdf · 실험
실 평가 LAB_TEST_GUIDE.md

HD_ACS 설치 매뉴얼 v1.0 · 2026-09-02 · 원본 truth는 docs/INSTALL_MANUAL.md